

os161_abc

Your portal to resources on OS161 at UBC.

[Home](#)[OS161 installation](#)[Understanding and Debugging](#)[OS161 Manual](#)[Contact Us](#)

Build OS161

Conventions

This document assumes you are working on OS/161 in a directory `~/os161` (immediately under your home directory) and that you have a source tree in `~/os161/src`. If you are working in a somewhere else or have arranged things differently, make the appropriate substitutions.

Configure the tree

The first step is to run the configure script in the top-level directory of the source tree. This sets things up to work with the OS you are working on and tells the makefiles what to do with certain things.

Use the `--ostree` option to tell it where you want the system to be installed. A good place for this (given the directory assumptions above) is `~/os161/root`. Then you would run:

```
cd ~/os161/src
./configure --ostree=$HOME/os161/root
```

Note that you need `$HOME` rather than just `~` because the shell won't expand the tilde when it's in the middle of an argument.

The script creates a file `defs.mk` that holds definitions used by the makefiles. You can edit this manually if necessary, although any such edits will be lost if you rerun the script. Do not commit this file to your version control system, because it will be different for different source trees. Instead, tell the version control system to ignore it. (This can be done with a `.gitignore` or similar control file; consult your version control system's documentation.)

Rerun this step if you move your source tree to a different computer that is running a different OS.

Build userland

To build the C library and user programs (including the test programs and everything else) just type `bmake` in the top level of the source tree. When this completes, type `bmake install` to copy the results into `~/os161/root`.

You can compile a single program by typing `bmake` in its source directory; typing `bmake install` will install just that program into the right place in `~/os161/root`.

The makefiles support the other conventional make rules; `bmake depend` updates the header file depend information, `bmake clean` erases the compiled objects, and `bmake distclean` returns the tree to a fully pristine state. You can also run `bmake includes` from the top level (or from the kernel or any library) to update the

Useful Links

[Build OS161](#)[Debug OS161](#)[Browse source code](#)[How to use git](#)

header files used by the build process.

If you are on a multiprocessor or multicore machine, you can speed the build by doing a parallel make. Use the `-j` option to `bmake`: `bmake -j4`. The number is the maximum number of parallel jobs to run at once. Currently the best rule of thumb for this seems to be twice the number of cores you have.

If you are working on NetBSD, type `make` instead of `bmake`, as `bmake` is just a repackaged version of NetBSD's system make (BSD make) and neither you nor your course staff should bother installing an extra copy. (The builtin make on OpenBSD, FreeBSD, and/or Dragonfly may also be able to build OS/161. Any positive or negative feedback on this would be welcome.)

All the build results are created in the build directory under the top level of the source tree. This can be placed elsewhere if desired, such as on a RAM disk, or a local disk if your working tree is a network volume. Read the file `mk/os161.config.mk` to learn about the things you can set in `defs.mk` to adjust how the build works.

Rerun this step if you change any of the kernel headers exported to userland (the ones in `src/kern/include/kern` or `src/arch/*/include/kern`) or if you make changes to the userlevel libraries. If you make changes to a single program, it is sufficient to recompile and restall just it.

Configure a kernel

Historically, most Unix systems would run kernels that had been customized for a particular piece of hardware. Such kernels would have only the device drivers needed for that system, and would include only the kernel features that were going to be used. (Leaving out everything else saves memory.) They would also sometimes have custom tuned settings for things like scheduler or pageout behavior.

OS/161 has a similar mechanism. The kernel configurations are kept in the directory `src/kern/conf` and are by convention named with all caps. You can copy one of the configs and edit it to remove drivers that you aren't using, in the traditional way; however, with System/161's simple hardware there's not much point to that, so you will probably just use the standard configurations. Choose the one you currently want to build. (The one you will use when first starting off is probably called either DUMBVM or ASST0.)

Now run the OS/161 config program. This is a shell and awk script, and it lives in the same directory as the config files. Assuming you mean to configure DUMBVM you run:

```
cd kern/conf
./config DUMBVM
```

Note the `./` to run the script out of the current directory. If you leave this off, you will probably end up running your host OS's kernel config tool, which probably will not like OS/161's kernel config files very much. The config script creates a kernel compile directory for the configuration you choose. The kernel is compiled there.

Rerun this step if you change the kernel config, add new source files to the build, or add new build options.

Compile a kernel

To compile a kernel, first move to its compile directory:

```
cd $HOME/os161/src/kern/compile/DUMBVM
```

Then run first `bmake depend`, then `bmake`. Run `bmake install` to copy the kernel to `~/os161/root` where you can boot it in System/161. If this fails, make sure your `ostree` directory exists; if it does, the most likely problem is that you forgot to run configure at the top level to create `defs.mk`.

Remember to install your newly built kernel before trying to test it. (The reason builds are numbered and the kernel prints the build number when booted is so you can easily tell if the kernel you just ran is really the one

you just compiled.)

The conventional `clean` and `distclean` targets will do the expected thing, if you need to rebuild the kernel from scratch.

Rerun `bmake` depend if/when you change header file inclusions, or after re-running `config`. Otherwise, there is no need to repeat it. Rerun `bmake` to recompile as needed.

Run the kernel

The install process will have created a `root` directory inside your OS161 base directory. In other words, your `root` directory will be at the same level as your `src` directory.

```
cd $HOME/os161/root
```

You will run `os161` on top `sys161` -- a MIPS hardware simulator. So first you need to get a configuration file for `sys161`.

```
wget https://people.ece.ubc.ca/~os161/download/sys161.conf.sample -O sys161.conf
```

Now run your kernel:

```
sys161 kernel
```

[HOME](#) | [INSTALL OS161](#) | [UNDERSTAND AND DEBUG OS161](#) | [CONTACT US](#)